Car Insurance API Requirements

1. Project Goal

The API should allow users to:

- See cars and their owners
- Add insurance policies for cars
- Check whether a car is insured on a specific date
- Register claims for cars
- View a car's insurance history

2. Scope

2.1 Required Scope

The project must include:

- FastAPI application
- Postgresql database
- SQLAlchemy models
- Pydantic request and response schemas
- Basic validation
- Basic error handling
- Simple service functions
- A small set of API endpoints
- Logging

2.2 Additional Scope – if time allows

- Authentication and authorization
- APScheduler background jobs
- Advanced logging with `structlog`
- Request ID middleware
- Complex ownership history
- Enforcing overlapping policies at the database level
- Test coverage

3. Learning Goals

- Create a FastAPI project
- Define SQLAlchemy ORM models
- Create relationships between database tables
- Use Postgresql for local development
- Use Pydantic schemas for API input and output
- Build REST endpoints
- Validate request data
- Return useful error messages
- Separate API routers from business logic

---

4. Simplified Business Requirements

4.1 Core Concepts

Owner

An owner represents a person who owns one or more cars.

Each owner has:

- `id`
- `name`
- `email`
- `birthdate`
- `driver license category`
- `year of getting the driver license`

Car

A car represents a vehicle.

Each car has:

- `id`
- `vin`
- `make`
- `model`
- `year_of_manufacture`

- owner_id
- power
- cc
- category

Each car belongs to exactly one owner.

Insurance Policy

An insurance policy represents insurance coverage for one car.

Each policy has:

- id
- car_id
- provider
- start_date
- end_date
- paid_amount
- status

A policy is valid on a date when:

```
start_date <= requested_date <= end_date
```

Claim

A claim represents a reported insurance claim for a car.

Each claim has:

- id
- car_id
- claim_date
- description
- amount
- created_at
- (image)

5. Functional Requirements

5.1 Health Check

Endpoint

```
GET /health
```

Response

```
{
  "status": "ok"
}
```

5.2 List Cars with Owners

Endpoint

```
GET /api/cars
```

Description

Return all cars with their current owner information.

Example Response

```
[
  {
    "id": 1,
    "vin": "WVWZZZ1JZXW000001",
    "make": "VW",
    "model": "Golf",
    "yearOfManufacture": 2018,
    “power”: 170,
    “cc”: 1995,
    “category”: “euro 3”,
    "owner": {
      "id": 1,
      "name": "Alice Smith",
      "email": "alice@example.com"
    }
  }
]
```

5.3 Create an Insurance Policy

Endpoint

```
POST /api/cars/{carId}/policies
```

Description

Create a new insurance policy for an existing car.

Request Body

```
{
  “provider”: "AXA",
  “startDate”: "2025-01-01",
  “endDate”: "2025-12-31",
  “paid_amount”: 1230.00
}
```

Rules

- `carId` must belong to an existing car.
- `startDate` is required.
- `endDate` is required.
- `endDate` must be the same as or after `startDate`.
- Dates must use format `YYYY-MM-DD`.

Success Response

Status:

```
201 Created
```

Example:

```
{
  "id": 1,
  "carId": 1,
  "provider": "AXA",
  "startDate": "2025-01-01",
  "endDate": "2025-12-31",
  "paid_amount": 1230.00,
  "status": "ACTIVE"
}
```

Error Examples

If the car does not exist:

```
{
  "detail": "Car not found"
}
```

If the end date is before the start date:

```
{
  "detail": "endDate must be greater than or equal to startDate"
}
```

5.4 Check Insurance Validity

Endpoint

```
GET /api/cars/{carId}/insurance-valid?date=2025-06-01
```

Description

Check whether a car has at least one valid insurance policy on a given date.

Success Response

```
{
  "carId": 1,
  "date": "2025-06-01",
  "valid": true
}
```

Rules

- `carId` must belong to an existing car.
- `date` must use format `YYYY-MM-DD`.

- The date should be a realistic year, for example between `1900` and `2100`.

Validity Rule

A policy is active when:

```
policy.start_date <= date <= policy.end_date
```

5.5 Register a Claim

Endpoint

```
POST /api/cars/{carId}/claims
```

Description

Create a new claim for an existing car.

Request Body

```
{
  "claimDate": "2025-03-10",
  "description": "Rear bumper damage",
  "amount": 450.00
}
```

Rules

- `carId` must belong to an existing car.
- `claimDate` is required.
- `description` is required and cannot be empty.
- `amount` must be greater than `0`.
- `amount` should be reasonable, for example less than `1,000,000`.

Success Response

Status:

```
201 Created
```

Example:

```
{
  "id": 1,
  "carId": 1,
  "claimDate": "2025-03-10",
  "description": "Rear bumper damage",
  "amount": 450.00
}
```

5.6 Retrieve Car History

Endpoint

```
GET /api/cars/{carId}/history
```

Description

Return all insurance policies and claims for a car, ordered by date.

Example Response

```
[
  {
    "type": "POLICY",
    "policyId": 1,
    "startDate": "2025-01-01",
    "endDate": "2025-12-31",
    "provider": "AXA"
    “paid_amount”: 1230.00,
    “status”: “INACTIVE”
  },
  {
    "type": "CLAIM",
    "claimId": 1,
    "claimDate": "2025-03-10",
    "amount": 450.00,
    "description": "Rear bumper damage"
  }
]
```

Rules

- `carId` must belong to an existing car.
- Items should be ordered chronologically.
- For policies, use `startDate` for ordering.
- For claims, use `claimDate` for ordering.

---

6. Validation Requirements

6.1 Dates

Dates must:

- Use format `YYYY-MM-DD`
- Be valid calendar dates
- Have a year between `1900` and `2100`

Examples of valid dates:

```
2025-01-01
2030-12-31
```

Examples of invalid dates:

```
2025-99-99
01-01-2025
1800-01-01
2200-01-01
```

6.2 Policy Validation

Each policy must have:

- `startDate`
- `endDate`
- `carId`
- `paidAmount`
- Optional `provider`

Validation rule:

```
endDate >= startDate
1900 <= endDate,startDate <= 2100
0 < paidAmount <= 1000000
1 <= provider length <= 100 if not null
Provider must contain only letters and numbers and separated only by one space
```

Prevent overlapping policies.

6.3 Claim Validation

Each claim must have:

- `claimDate`
- `description`
- `amount`

Validation rules:

```
0 < amount <= 1000000
1 <= description lenght <= 2000
1900-01-01 <= claimDate <= today
```

---

7. Simplified Technical Requirements

7.1 Runtime

Use:

```
Python 3.11+
```

7.2 Required Libraries

Use the following main libraries:

```
fastapi
uvicorn
sqlalchemy
pydantic
```

Optional but useful:

```
python-dotenv
pytest
```

7.3 Database

Use PostgreSQL.

---

8. Suggested Project Structure

```
app/
  main.py

  api/
    routers/
      cars.py
      policies.py
      claims.py
      history.py
      owner.py
      health.py
    schemas/
      car_schemas.py
      owner_schemas.py
    deps.py

  db/
    database.py
    models.py

  services/
    policy_service.py
    claim_service.py
    validity_service.py
    history_service.py

   repositories/
    car_repository.py
    claim_repository.py
    .......

  utils/
    dates.py

tests/
  test_claims.py
```

---

9. API Endpoints Summary

Method	Endpoint	Description
GET	`/health`	Check if API is running
POST	`/api/owners`	
GET	`/api/owners`	Filter on : category
GET	`/api/owners/{owner_id}`	
PATCH	`/api/owners/{owner_id}`	

DELETE	`/api/owners/{owner_id}`	
GET	`/api/cars`	List cars with owners (make,model,category,owner_id)
POST	`/api/cars`	Create a car
DELETE	`/api/cars/{car_id}`	Delete a car
POST	`/api/cars/{carId}/policies`	Create policy for car
GET	`/api/cars/{carId}/insurance-valid?date=YYYY-MM-DD`	Check car has insurance valid ar a specific date
POST	`/api/cars/{carId}/claims`	Register claim
GET	`/api/cars/{carId}/history`	Get car policy and claim history. Filter on: type
GET	`/api/policies`	Get all policies(+- filters)
GET	`/api/licenses`	Returns available driver license categories
GET	`/api/cars/{car_id}`	Get details about 1 car
GET	`/api/policies/active-policy?car_id={car_id}`	Gets the current active policy for a car.

13. Suggested Implementation Order

- Endpoints on owners, cars, policies, claims, history.

---

14. Bonus Tasks

These are optional and should only be attempted after the required version works.

Bonus 1: Prevent Overlapping Policies

When creating a new policy, check whether it overlaps with an existing policy for the same car.

Two policies overlap when:

```
new_start <= existing_end AND new_end >= existing_start
```

If they overlap, return an error.

Bonus 2: Add Basic Tests

Add unit tests (|| integration tests) for:

- `/health`
- Listing cars
- Checking insurance validity
- Creating a claim
- Getting car history

Bonus 3: Add Background Expiry Logging

Add a simple function that finds expired policies and logs them.

Expected log message:

```
Policy 1 for car 1 expired on 2025-12-31
```

To avoid duplicate logs, add a field to the policy table:

```
logged_expiry_at
```

---

15. Acceptance Criteria

The project is complete when:

1. The FastAPI app starts successfully.
2. `/health` returns `{"status": "ok"}`.
3. `/api/cars` returns cars with owner information.
4. A policy can be created for an existing car.
5. Creating a policy for a non-existent car returns `404`.
6. A policy with `endDate` before `startDate` is rejected.
7. Insurance validity returns `true` when a policy covers the requested date.
8. Insurance validity returns `false` when no policy covers the requested date.
9. A claim can be created for an existing car.
10. A claim with an empty description or invalid amount is rejected.
11. `/api/cars/{carId}/history` returns policies and claims in chronological order.
12. The API is usable from FastAPI Swagger UI at `/docs`.
